

Prompt caching

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 9 of 12 · English notes

Without cache, every Messages call is **stateless**: the PDF, the huge system prompt, the thinking over that PDF — all discarded after the response. The next click pays input (and thinking) again. **Prompt caching** stores a prefix on Anthropic’s side for a short TTL (class: **5 minutes** or **1 hour**) so later requests reuse it at cache-hit prices. Vijay also talked about **container ids** so follow-ups stick to the replica that holds the cache.

Why it exists

Chat-with-PDF is the motivating story: converting a million-token document and sending it on every question is an input invoice each time. Cache the document (and stable instructions). Subsequent user questions are small; you pay a write once, then hits. Combining cache with extended thinking without cache was described as double waste: re-read + re-think.

How the lab enabled it

On the system content (or other cacheable blocks in current APIs), set `cache_control` with type: `ephemeral`. That flag marks “please cache this prefix.” Ephemeral means the server remembers only for the agreed duration, then discards — Vijay tied that to SLA language: not a permanent store, not your RAG database.

The demo mixed a large “code prompt” / HTML-ish blob as system content (and briefly tangled with tool calling when the notebook went sideways). Conceptually: put the **large invariant** behind `cache_control`; leave the per-turn user question after the breakpoint.

Write, hit, read — and money

Operation (class)	Meaning	Price talk on the slide (illustrative)
Cache write	First time that prefix is stored	Higher than raw input; 5-minute vs 1-hour TTLs priced differently (e.g. ~\$12.50 vs ~\$20 / MTok on that deck)
Cache hit	Later request reuses the prefix	Much cheaper (on the order of ~\$1 / MTok on that deck)
Cache read	Reading cached tokens as part of processing	May be bundled or billed depending on model — check live pricing

Exact dollars from 17–20 August 2026 are **not** exam constants. The structure is: write is dear, hit is cheap, output is still output, thinking tokens still exist if enabled. See current prompt-caching and pricing pages.

Container id and sticky routing

Class architecture: horizontally scaled LLM containers. A cache lives on the instance that wrote it. The Message response can include a **container** identifier (already glimpsed on 17 August). Send it back on the next request so you land on the same cache. If you spray traffic across random replicas, you pay writes again. Official docs may describe automatic prefix matching instead of manual container pinning — if they disagree with the notebook, **docs win**, but be ready to explain Vijay’s replica story.

What cache is not

- Not Chat session memory in the product sense — it expires.
- Not RAG / a SQL database you query (17 August Q&A).
- Not a substitute for resending the *changing* user turn.
- Not Claude Pro’s five-hour usage refresh.

Design rules

1. Stable, large prefix first (system, tools, documents).
2. Mark `cache_control` at the breakpoint.
3. Variable question last.
4. Reuse the same model id; changing model usually misses the cache.
5. Watch usage fields: `cache_creation` vs `cache_read` tokens in the response.

MINIMUM TOKENS AND TTL NAMES CHANGE

Today’s APIs document minimum prefix sizes and TTL enums. The classroom word was “ephemeral” plus 5 minutes / 1 hour. Align with **prompt caching** before you sit the exam.

Handoff to tools and MCP

The rest of 20 August introduced tools/MCP as the way to inject **real-time and proprietary** data — another context-enhancement path, examined in module 10. Cache keeps *static* context cheap; tools fetch *fresh* context.

Official documentation

• Prompt caching

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

`cache_control`, TTLs, breakpoints, usage fields.

• Messages API

<https://docs.claude.com/en/api/messages>

Where `cache_control` sits on system and content blocks.

• Extended thinking

<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>

Do not casually re-think a cached PDF every turn.

- **PDF support**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Documents are the usual large prefix to cache.

- **Rate limits**

<https://docs.claude.com/en/api/rate-limits>

Throughput still applies on cache hits.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.